Face Detection in Unconstrained Environments: A Comparative Analysis of Traditional and Deep Learning Models

Eduardo Alves[1 [0009−0003−7670−9198]], Francisco Carvalho[2 [0009−0006−3119−7055]], and Orlando Marinheiro[3 [0009−0001−3083−6226]]

Department of Electronics, Telecommunications and Informatics (DETI), University of Aveiro, Aveiro, Portugal
Artificial Intelligence Applications — Course Instructors: Sónia Gouveia, Ana Martins
{eduardoalves@ua.pt, francisco.joao.lopes.carvalho@ua.pt, orlandomarinheiro@ua.pt}

Abstract. Face detection is a foundational task in computer vision, serving as a critical precursor for facial analysis applications such as recognition, alignment, and tracking. While traditional methods like the Viola-Jones algorithm provide high computational efficiency, they often struggle under unconstrained real-world conditions characterized by scale variations, occlusions and poses. This study evaluates the performance of the classical Haar Cascade classifier against a challenging benchmark dataset, contrasting its capabilities with modern deep learning-based architectures, like VGG-16 and YOLO. Our empirical findings validate the structural limitations of traditional cascade detectors in unconstrained environments, aligning with established benchmarks in literature.

Keywords: Face Detection · Deep Learning · Haar Cascade · Faster R-CNN · VGG-16 · YOLO · Hyperparameter Optimization · Inference Latency · WIDER FACE

1 Introduction

Face detection is an essential initial step for a wide range of computer vision and facial analysis systems, including face alignment, facial expression recognition, and biometric verification. Given an arbitrary image, the objective of face detection is to determine whether human faces are present and, if so, to return their exact spatial location and extent within the image.

Although humans perform this task effortlessly, building robust automated systems remains challenging. Real-world scenarios frequently introduce severe visual variations, such as extreme head poses, low resolution, occlusions, and extreme lighting conditions.

Historically, real-time face detection was revolutionized by classical cascade classifiers, which enabled efficient processing for near-frontal faces. However, as modern applications transition into complex, unconstrained environments, these traditional methods often suffer from significant performance degradation.

This paper presents a systematic comparative evaluation between classical machine learning and modern deep learning paradigms for face detection in unconstrained scenarios. Specifically, we investigate the practical boundaries and performance limits of the classical Viola-Jones Haar Cascade classifier by contrasting its efficiency and accuracy against two distinct neural network frameworks: a two-stage Faster R-CNN utilizing a VGG-16 backbone (optimized via Optuna) and several modern single-stage detection architectures from the YOLO family. By assessing all paradigms on a standardized benchmark, this work bridges the gap between traditional lightweight approaches and contemporary deep learning solutions, analyzing the trade-offs between computational overhead, inference latency, and detection robustness under severe real-world variability.

2 State of the Art

Face detection methodologies have evolved significantly over the decades, transitioning from traditional geometric and appearance-based heuristics to modern data-driven deep learning frameworks. In literature, face detection algorithms are typically categorized into four main streams: cascade-based methods, part-based models, channel feature-based methods, and neural network-based approaches.

To evaluate the trade-off between precision and computational efficiency, recent studies often reference standard benchmarks such as the WIDER FACE dataset [6], which categorizes challenges by scale, occlusion, and illumination severity. This work explores this technological evolution by analyzing three distinct paradigms: a classical machine learning method, a traditional convolutional neural network (CNN), and a modern single-stage detection architecture.

2.1 Classical Machine Learning: Haar Cascades

The Haar Cascade classifier acts as a baseline for computational efficiency. While providing rapid execution, its reliance on handcrafted Haar-like features and a boosted cascade structure makes it fundamentally vulnerable to severe unconstrained conditions. Comparative studies, such as the baseline evaluations established by Naser *et al.* [3], highlight the degradation of classical cascades under facial occlusions and extreme pose variations, scenarios prevalent in unconstrained environments.

2.2 Traditional Convolutional Neural Networks (VGG16)

To understand the computational weight of non-specialized deep architectures, traditional CNNs such as VGG16 have been adapted for face detection tasks, commonly integrated with region proposal frameworks like Faster R-CNN [2]. Although deep representations capture complex facial patterns, direct application to unconstrained benchmarks often exposes structural limitations, particularly when dealing with microscopic scales or heavy background clutter, while demanding significant training durations and computational overhead [2].

2.3 The Modern State of the Art: YOLO Architectures

Modern object detection includes single-stage architectures such as the YOLO (You Only Look Once) family [5], which combines localization and classification within a single network. YOLO26n is the lightweight nano variant of YOLO26, designed to balance detection performance and temporal computational cost. Its COCO-pretrained weights provide a general-purpose baseline, while fine-tuning on WIDER FACE adapts the model to face detection. Specialized implementations include YOLOv8n-Face from Lindevs and YOLO26n-Face from akanametov, both providing checkpoints trained on WIDER FACE. These approaches motivate comparing general-purpose YOLO26n, its fine-tuned version, and the two specialized face detectors in terms of detection quality and inference speed [1].

2.4 Quantitative Comparative Analysis

To contextualize our performance baselines, Table 1 summarizes representative literature results across varying levels of difficulty (Easy, Medium, and Hard subsets).

Table 1. Overview of representative face detection baseline performance on unconstrained WIDER FACE benchmarks (Average Precision).

Method / Architecture	Reference	Easy	Medium	Hard
Viola-Jones (Haar Cascades)	Yang *et al.* [6]	41.2%	33.3%	13.7%
Faster R-CNN / VGG16	Jiang *et al.* [2]	89.8%	87.9%	81.2%
YOLO5Face	Qi *et al.* [4]	95.0%	94.1%	89.2%
YOLO-FaceV2	Zhang *et al.* [7]	95.8%	94.6%	90.1%

2.5 Project Positioning and Expectations

Grounded in this bibliographic overview, our research investigates the practical trade-offs between precision and computational resource consumption. We anticipate the following outcomes:

- **Computational Cost (Time-Centric Analysis):** Computational cost is assessed through training duration and inference latency. Traditional methods such as Haar Cascades require no additional model training, whereas deep learning architectures involve longer training processes. Inference time is measured in milliseconds per image to compare the computational efficiency of the evaluated models.
- **Inference Speed:** Modern single-stage detectors (YOLO) offer superior real-time inference speeds compared to heavier traditional VGG backbones, frequently outperforming even CPU-bound cascades in densely populated scenes due to their predictive single-stage nature.
- **Robustness:** While lightweight classical methods remain viable under constrained, near-frontal conditions, modern optimized single-stage networks establish the benchmark for handling scale variance and occlusion without sacrificing temporal throughput.

3 Methodology

3.1 Dataset Evaluation

To evaluate the performance and behavior of the face detection models, dataset composition and integrity must be carefully analyzed prior to experimentation. In this work, the primary benchmark utilized is the **WIDER FACE** dataset, renowned for its extreme variability in scale, pose, and occlusion.

The official structure of WIDER FACE comprises distinct partitions: a training set (12, 880 images with 156, 994 valid faces), a validation set (3, 226 images with 39, 112 valid faces), and a test set (16, 097 images). A rigorous cross-validation against local storage confirmed the structural integrity of the files, verifying zero corrupted images across the official splits. This dataset also incorporates difficulty tags based on blur, occlusion levels, and crowding density.

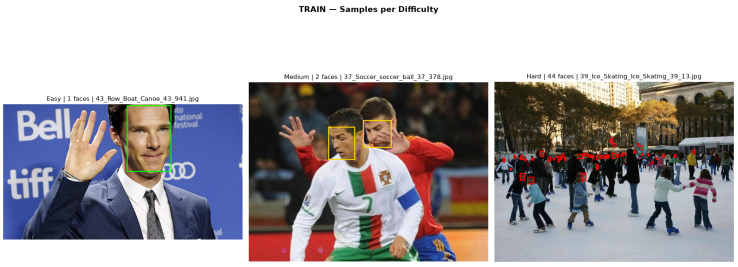


Fig. 1. Sample images of each difficulty tier from the WIDER FACE dataset.

A methodological limitation encountered during this work is that ground-truth annotations for the official WIDER FACE test partition are withheld by its creators. To overcome this limitation and enable an independent evaluation, an annotated external dataset was adopted as our testing partition. Specifically, we utilized the Kaggle Face-Detection-Dataset provided by Fares Elmenshawii. Its original training and validation folders (containing 13, 386 and 3, 347 images, respectively) were combined to form a single, robust evaluation set consisting of 16, 733 images and 51, 846 annotated faces.

A comparative statistical evaluation reveals distinct structural differences between the training domain (WIDER FACE) and the evaluation domain (External Test Set). As evidenced by Figure 4, the external dataset features a significantly lower face density per image (mostly ranging between 1 and 3 faces), whereas WIDER FACE frequently handles complex crowd scenes with numerous subjects. Furthermore, Figure 3 demonstrates that the relative dimensions (width and height ratios) of the faces are shifted toward higher values in the external dataset, indicating larger, more prominent subjects.

While these characteristics render the external test set less challenging regarding extreme crowd density and micro-scale detection, it serves as a valuable out-of-sample benchmark. It allows us to assess the models' generalization capabilities under more controlled framing constraints, complementing the severe unconstrained challenges present in the WIDER FACE validation subset.

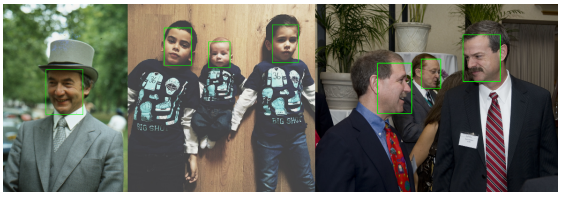


Fig. 2. Representative sample images from the Kaggle external test set with ground-truth bounding boxes, illustrating lower face density and larger relative face dimensions compared to WIDER FACE.

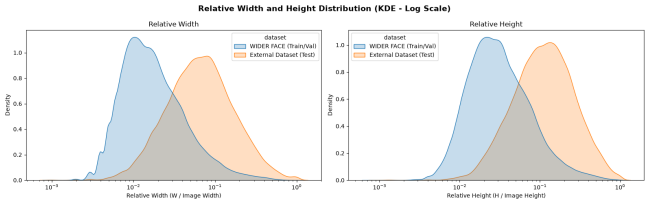


Fig. 3. Relative width and height distributions of the bounding boxes, showing that faces in the external dataset occupy a larger proportion of the frame.

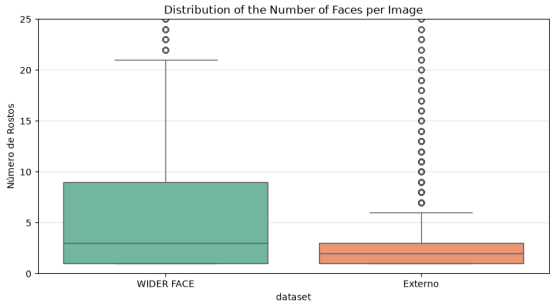


Fig. 4. Distribution of the number of faces per image across both datasets (Y-axis truncated at 25 for visual clarity, omitting extreme outliers).

3.2 Evaluation Protocol and Metrics

To ensure a fair and controlled out-of-sample evaluation across all architectures, all final inference testing was executed on a single local workstation. The hardware environment comprised an Intel Core i5-13450HX CPU, 16 GB of RAM, and an NVIDIA GeForce RTX 4060 GPU (8 GB VRAM).

Regarding the data pipeline, all learnable models (VGG-16 and YOLO) were exclusively trained and validated on the WIDER FACE dataset partitions. Hyperparameter optimization and model selection were also performed exclusively using these partitions. Subsequently, to assess generalization, the final evaluation of all models, including the

pre-trained Viola-Jones classifier, was conducted on the external Kaggle test set, which remained entirely independent throughout the development process.

The comparative analysis was standardized around three primary metrics:

- **Mean Average Precision (mAP@0.50):** To quantify overall detection accuracy.
- **Precision-Recall Curves:** To visualize the trade-off between false positives and false negatives across continuous confidence thresholds.
- **Inference Latency:** Measured in milliseconds (ms) per image to evaluate the computational efficiency and real-time viability of each architecture.

3.3 Viola-Jones Algorithm (Haar Cascade)

The classical Viola-Jones framework was employed utilizing the pre-trained Haar Cascade frontal face classifier provided by OpenCV. Unlike learnable deep architectures, this classical model relies on a fixed, pre-optimized boosted cascade of weak classifiers based on handcrafted Haar-like features and integral images. The architecture operates by evaluating sliding windows across an image pyramid, passing them through a sequence of increasingly complex AdaBoost stages that reject background regions to maintain high computational efficiency.

Consequently, no direct training phase was executed on our target dataset. Instead, adaptation to the evaluation domain was achieved through hyperparameter optimization via a grid search on the validation partition. This search specifically tuned the *scaleFactor* (controlling the step size of the image pyramid), *minNeighbors* (dictating the required spatial consensus of overlapping detections), and *minSize* (acting as a bottom-up filter for distant or noisy features) to find the optimal balance between detection sensitivity and the suppression of false positives.

A critical methodological enhancement in our evaluation was the extraction of continuous confidence scores from this otherwise discrete classifier. While standard Haar Cascade implementations yield only binary bounding box coordinates, there is a method `detectMultiScale3` that returns raw AdaBoost stage weights (`levelWeights`). By flattening and assigning these continuous weights to each prediction, the classical algorithm's detections could be ranked by confidence. This allowed the framework to be evaluated using standard Non-Maximum Suppression (NMS) and facilitated the extraction of continuous confidence scores, a necessary step to compute the standardized metrics defined in our evaluation protocol.

Furthermore, recognizing that the Viola-Jones sliding-window approach fundamentally lacks the bounding box regression capabilities inherent to modern neural networks, an explicit spatial sensitivity analysis was integrated into the protocol. By sweeping the Intersection over Union (IoU) acceptance threshold, we systematically quantified the model's structural limitations in achieving pixel-perfect spatial alignment, isolating localization errors from pure classification failures.

3.4 Faster R-CNN (VGG-16) Architecture and Training

For the Faster R-CNN architecture, a transfer learning strategy was adopted using a VGG-16 backbone with pre-trained convolutional weights acting as the feature extractor. To

this backbone, a Region Proposal Network (RPN) and detection heads were integrated to perform bounding box regression and face classification.

Initially, the network was calibrated using baseline parameters from literature. To systematically improve detection performance, hyperparameter tuning was conducted using Optuna across 25 trials on Kaggle, with each candidate configuration trained and evaluated for 5 epochs. The objective value to minimize was defined as the total validation loss. Out of the 25 trials, 20 completed successfully, with Trial 24 achieving the lowest validation loss of 0.1613. The optimal configuration identified an Adam optimizer with a learning rate of 2.76×10^{-5}, an RPN NMS threshold of 0.50, medium anchor scales, and an L_2 regularization parameter (`weight_decay`) of 9.94×10^{-5}.

Figure 5 displays the optimization history (left) and the relative hyperparameter importance (right). The RPN Non-Maximum Suppression threshold (`rpn_nms_thresh`) was the most influential parameter, accounting for the highest impact on convergence, followed by the learning rate (`lr`) and weight decay.

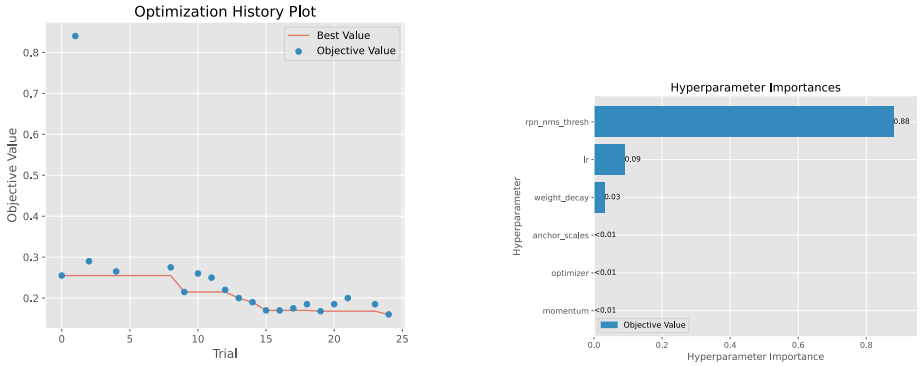


Fig. 5. Optuna optimization history minimizing validation loss (left) and relative hyperparameter importance (right) for Faster R-CNN with VGG-16 backbone.

The best configuration was subsequently trained locally for up to 30 epochs, incorporating early stopping (with a patience of 5 epochs) and L_2 weight decay to regularize the cost function and prevent overfitting. Training halted automatically at epoch 9 due to validation loss plateauing, requiring 7 hours, 34 minutes, and 29 seconds.

Figure 6 illustrates the progression of the total cost function (left) alongside the breakdown of the four individual Faster R-CNN loss components (right): classifier loss (`loss_classifier`), box regression loss (`loss_box_reg`), RPN objectness error (`loss_objectness`), and RPN proposal regression error (`loss_rpn_box_reg`).

3.5 YOLO Architectures

The YOLO comparison uses four checkpoints: YOLO26n base, YOLO26n fine tuned, YOLOv8n-Face and YOLO26n-Face. YOLO performs localisation and classification within a single network. The base model uses Ultralytics COCO-pretrained YOLO26n

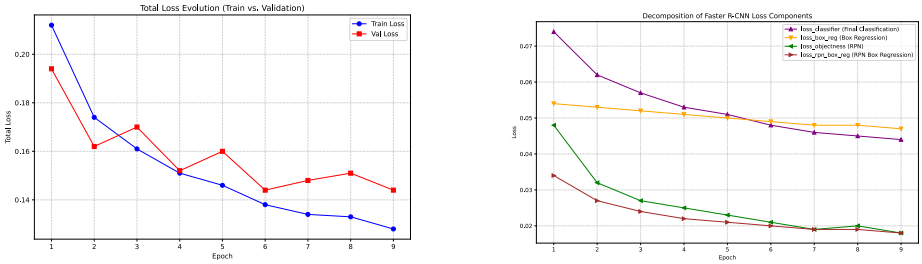


Fig. 6. Faster R-CNN (VGG-16) training loss trajectories: total loss comparison between training and validation splits (left) and decomposition of the four loss components (right).

weights; the fine tuned model starts from those same weights and learns the single class `face`. The external face checkpoints are the nano model distributed by Lindevs and the YOLO26n-Face model distributed by akanametov. They were used without additional training. The generic base model has no face class, so its `person` detections serve as a starting-point reference against the face annotations.

Dataset annotations were converted to normalized YOLO bounding box coordinates for a single `face` class, clipping box dimensions to image boundaries and filtering out invalid training annotations. Model performance was monitored on the validation split to select the optimal checkpoint without updating weights directly during validation cycles.

The 100-epoch training run was performed on Google Colab with an NVIDIA Tesla T4 GPU. The saved configuration records Ultralytics 8.4.181, image size 640, batch size 8, automatic mixed precision, seed 2 and the MuSGD optimiser. Early-stopping patience was 15 epochs, but all 100 epochs were completed. The setting `freeze: null` means that the backbone was not explicitly frozen; training was therefore not restricted to the final layers. Evaluation uses the validation-selected `best.pt` from the 100-epoch run.

Learning rates used three warmup epochs followed by linear decay, with `lr0`=0.01, `lrf`=0.01 and `cos_lr`=false. The MuSGD implementation applies a threefold learning-rate multiplier to selected parameter groups. The eight logged parameter groups followed two distinct learning-rate schedules. Groups with identical learning rates are represented by a single curve in Figure 7 (right). The evolution of the bounding box loss across epochs for both training and validation is illustrated in Figure 7 (left).

Training was resumed after epochs 65 and 89. The accumulated training and validation runtime across the three recorded segments was approximately 8 hours and 32 minutes. This excludes data preparation and interruptions between Colab sessions. The training times of the external checkpoints were not measured.

During inference on the local testing environment, all four YOLO models utilized an input size of 640×640, FP32 precision, and a batch size of 1. The inference parameters were strictly controlled: the minimum confidence threshold was set to 0.001, the Non-Maximum Suppression (NMS) IoU threshold was 0.70, and a maximum of 1,000 detections were permitted per image.

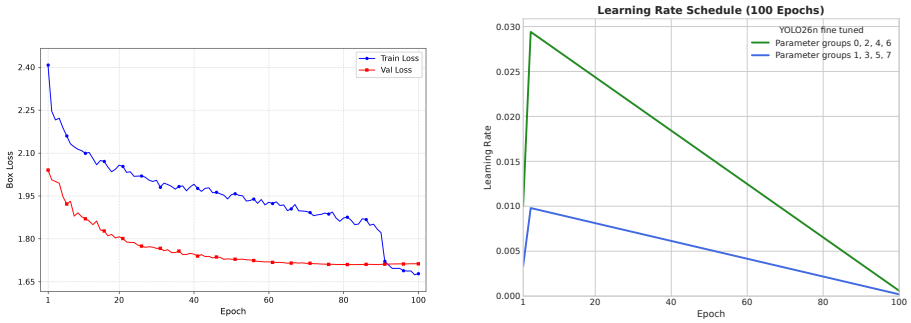


Fig. 7. YOLO26n fine-tuned training monitoring over 100 epochs on Google Colab with T4. Left: bounding box loss trajectory for training and validation splits. Right: learning-rate decay schedule across parameter groups.

4 Results

4.1 Viola-Jones Algorithm (Haar Cascade)

Detection Quality and Spatial Sensitivity. Figure 8 presents the quantitative evaluation of the Haar Cascade classifier on the test set. Unlike the deep learning models evaluated in this study, the classical Viola-Jones framework achieved a significantly lower Mean Average Precision (AP@0.50) of **0.2103 (21.03%)**.

As observed in the Standard Precision-Recall curve (Figure 8, left), the plot truncates abruptly around 0.30 Recall. This indicates that the model missed approximately 70% of the faces in the dataset (False Negatives), regardless of the confidence threshold. This behavior highlights the classical sliding-window's conservative nature and its inability to generalize to extreme variations in scale, pose, and occlusion.

Furthermore, the Spatial Alignment Sensitivity analysis (Figure 8, right) exposes the lack of bounding box regression. While performance remains stable at low IoU thresholds (0.1 to 0.4), which shows the model can reliably locate the general facial region, it collapses precipitously for IoU $\geq$ 0.6, almost reaching 0 for all metrics when near perfection (IoU $\geq$ 0.9) is required. This confirms that the rigid scaling mechanism of the Haar Cascade fundamentally fails to frame facial features with the pixel-level precision demanded by modern evaluation protocols.

Inference Latency. The Haar Cascade was executed entirely on the CPU recording a mean inference time of **107.82 ms** per image.

Qualitative Analysis. Visual inspection of the predictions provides critical context for the quantitative metrics, illustrating both the optimal capabilities and the dominant failure modes of the Viola-Jones architecture.

Figure 10 (left) demonstrates a best-case scenario. On a clear, frontal face, the model achieves excellent spatial alignment (IoU = 0.91). However, its susceptibility to

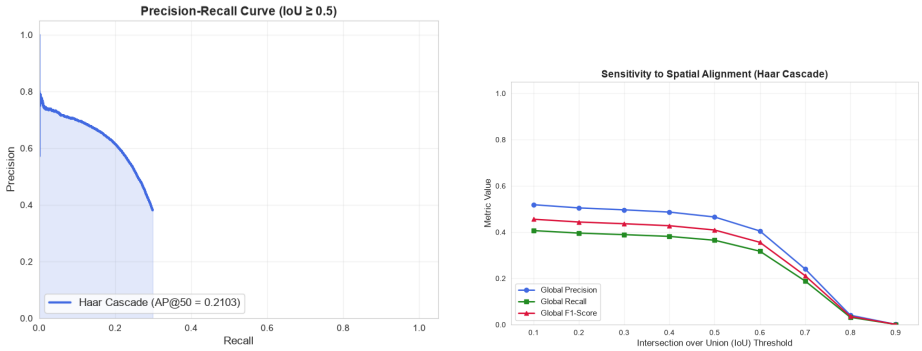


Fig. 8. Detection performance of the Haar Cascade classifier. Left: Standard Precision-Recall curve at IoU 0.50 (AP = 21.03%). Right: Sensitivity analysis across varying IoU thresholds, illustrating the collapse of spatial alignment metrics.

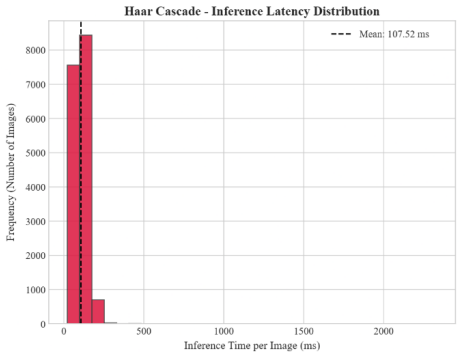


Fig. 9. Inference latency distribution (ms) per image for the Haar Cascade classifier executed on the CPU.

background hallucinations is still evident, as complex textures trigger a secondary False Positive detection.

Conversely, Figure 10 (right) illustrates a typical worst-case scenario: a structural False Negative. The algorithm entirely misses the profile face, highlighting its inability to generalize to severe pose variations since it was trained specifically on frontal Haar-like features.

4.2 VGG-16 Architecture Results and Comparison

To evaluate the actual impact of hyperparameter tuning, a comparative analysis was performed between the baseline version of the VGG-16 architecture (Faster R-CNN with standard literature parameters) and the version optimized via Optuna. The evaluation was conducted using two fundamental metrics: detection quality (measured through the



Fig. 10. Qualitative evaluation of the Haar Cascade predictions. Left (Best Case): Highly accurate frontal face detection (IoU = 0.91), accompanied by a False Positive hallucination near the wheelchair. Right (Worst Case): A complete False Negative demonstrating the model's inability to detect non-frontal profile faces. In both images, model predictions are denoted by dashed red bounding boxes, while ground truth annotations are shown in green (IoU $\geq$ 0.5) or yellow (IoU $<$ 0.5).

Precision-Recall curve and the Average Precision, AP) and inference latency per image (measured in milliseconds).

Figure 11 presents the quantitative results obtained on the test set.

As shown in the Precision-Recall curve (Figure 11, left), hyperparameter optimization yielded a substantial improvement in model precision. The Average Precision value (AP@0.50) increased from 0.709 (70.9%) in the baseline version to 0.799 (79.9%) in the optimized model, representing a direct gain of 9.0 percentage points. This improvement reflects the network's ability to maintain high precision levels even for Recall values above 0.6.

Additionally, the inference time analysis (Figure 11, right) demonstrates that this precision gain did not compromise computational efficiency. The optimized model exhibited a slightly more consistent latency distribution with lower outlier variance compared to the standard model, maintaining an average processing time stabilized around 70 ms per image.

4.3 YOLO

Detection quality. Figure 12 presents the results on the 16,733 Kaggle images. YOLO26n fine tuned achieved the highest mAP50 (84.63%). Compared with YOLOv8n-Face, this represents an improvement of 0.61 percentage points in mAP50. The three face detection checkpoints obtained similar mAP50 values.

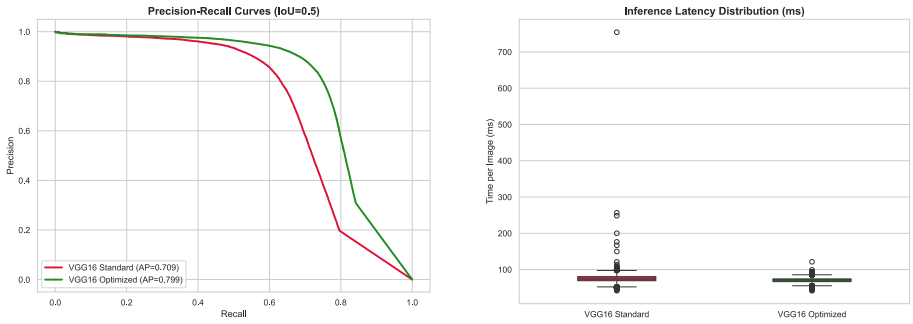


Fig. 11. Performance comparison between the VGG16 Standard and VGG16 Optimized models: Precision-Recall curves with AP values for IoU=0.50 (left) and inference latency distribution in ms per image (right).

The base model achieved only 0.17% mAP50 because its `person` detections were compared with face annotations. This result reflects the mismatch between the detection tasks and does not describe its performance on its original COCO task.

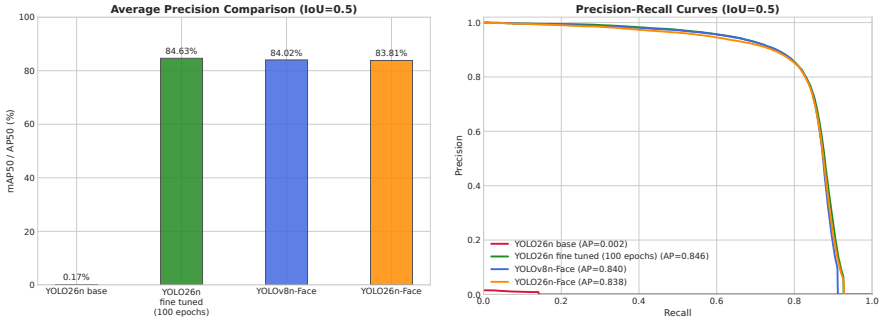


Fig. 12. Detection performance on Kaggle. Left: mAP50 comparison in percent. Right: precision–recall curves at IoU 0.50, with AP values expressed as proportions.

Inference latency and speed. YOLOv8n-Face achieved the lowest mean network inference time, 22.30 ms, corresponding to 44.84 FPS. YOLO26n fine tuned required 26.14 ms per image, corresponding to 38.26 FPS. Its mAP50 improvement of 0.61 percentage points over YOLOv8n-Face therefore came with 3.84 ms of additional mean network latency.

YOLO26n-Face had the highest mean network inference time, 35.62 ms. Its native inference also computes facial landmarks, although this evaluation scores only bounding boxes. Figure 13 summarises the timing measurements.

All models had higher mean than median latency. Figure 13 shows the variation between images, including all recorded outliers. Although YOLOv8n-Face had the lowest mean and median latency, the base model had the lowest P95. The ranking therefore depends on the timing statistic used.

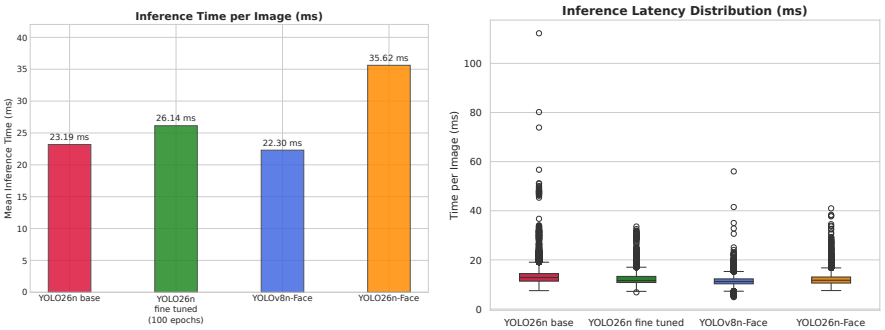


Fig. 13. Network inference latency on the local PC. Left: mean inference time per image. Right: latency distributions, showing medians, quartiles, whiskers at 1.5 times the interquartile range and all outliers.

4.4 Global Performance Comparison

To summarize the trade-offs between computational efficiency and detection robustness, Table 2 presents a unified comparison of all evaluated architectures.

Table 2. Global comparison of face detection models regarding detection quality (AP@0.50), inference speed, and hardware acceleration.

Model	Hardware	AP@0.50 (%)	Mean Inference Time (ms)
Viola-Jones (Haar Cascade)	CPU	21.03	107.82
VGG-16 (Standard)	GPU	70.90	74.18
VGG-16 (Optimized)	GPU	79.90	69.85
YOLO26n (Base)	GPU	0.17	23.19
YOLOv8n-Face	GPU	84.02	22.30
YOLO26n (Fine-tuned)	GPU	84.63	26.14
YOLO26n-Face	GPU	83.81	35.62

The results highlight the modern single-stage YOLO architectures as the optimal solution for unconstrained environments, significantly outperforming both the classical CPU-bound Haar Cascade and the heavier VGG-16 backbone in both accuracy and processing speed.

5 Conclusions

In this study, we addressed the complex problem of face detection in unconstrained environments, a challenge heavily characterized by severe scale variations, occlusions, and extreme poses. While traditional heuristics like the Viola-Jones Haar Cascade offer straightforward deployment and computational efficiency on standard CPUs, our empirical evaluation highlighted its structural inability to generalize to complex real-world conditions. This limitation was evidenced by a low Average Precision (21.03%) and a rapid degradation in spatial alignment accuracy at higher IoU thresholds.

Conversely, data-driven deep learning models demonstrated the representational robustness required to handle such complexities. The two-stage Faster R-CNN with a VGG-16 backbone provided strong detection capabilities, particularly after a systematic hyperparameter optimization via Optuna, which significantly improved its AP to 79.90%. However, the modern single-stage YOLO architectures emerged as the definitive optimal solution. The fine-tuned YOLO26n model achieved the highest overall accuracy (84.63% mAP@0.50), while YOLOv8n-Face offered the most efficient inference latency (22.30 ms per image). Ultimately, modern single-stage detectors successfully bridge the gap between high precision and real-time processing, establishing themselves as the standard for unconstrained face detection.

Author Contributions The workload was distributed among the authors as follows: **Eduardo Alves** led the configuration, fine-tuning, and evaluation of the YOLO architectures; **Francisco Carvalho** conducted the configuration, testing, and spatial sensitivity analysis of the Viola-Jones Haar Cascade algorithm, alongside the initial dataset statistical evaluations. **Orlando Marinheiro** was responsible for the implementation, training, and hyperparameter optimization of the VGG-16 (Faster R-CNN) architecture; All authors contributed equally to the writing and formatting of the final report.

References

1. Chen, W., Huang, H., Peng, W., et al.: Yolo-face: a real-time face detector. The Visual Computer **37**, 805–813 (2021)
2. Jiang, H., Learned-Miller, E.: Face detection with the faster r-cnn. In: 12th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2017). pp. 650–657. IEEE (2017)
3. Naser, O.A., Mashohor, S., Samsudin, K., et al.: Comparative analysis of mtcnn and haar cascades for face detection in images with variation in yaw poses and facial occlusions. Journal of Communications Software and Systems (2025)
4. Qi, D., Tan, W., Yao, Q., Liu, J.: Yolo5face: Why reinventing a face detector. arXiv preprint arXiv:2105.12931 (2021)
5. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 779–788 (2016)
6. Yang, S., Luo, P., Loy, C.C., Tang, X.: Wider face: A face detection benchmark. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 5525–5533 (2016)
7. Zhang, X., et al.: Yolo-facev2: A scale and occlusion aware face detector. arXiv preprint arXiv:2208.02019 (2022)